Exploring Cryptocurrency Trading Strategies

A dissertation submitted to the University of Manchester for the degree of Master of Science in the faculty of Science and Engineering

10217622

2018

Abstract

Trading cryptocurrencies is a very risky strategy for making a profit. Cryptocurrencies have been known to be extremely volatile in nature. This is due to the market manipulation that occurs with cryptocurrencies because of little to no regulation which causes rapid price movements throughout the market.

In this paper, I will be analyzing different types of trading strategies that range from making use of technical analysis, sentiment analysis, machine learning and much more. The strategies will be implemented in code to allow the use of automated trading.

From the results shown, it is possible that some trading strategies can be used to make a profit, however, results can be subject to change in certain market conditions. The paper shows that automated trading can be compatible with the cryptocurrency market.

List of Figures

1 Bitcoin price chart in 2017/2018 14
2 Pump and dump of a low market cryptocurrency 15
3 An example of a sell wall to restrict price increase 16
4 Characteristics of candlesticks 19
5 Latest candlestick charts of Bitcoin taken at 5th September 2018 19
6 Prices between different markets of NANO/BTC 22
7 Example of tri-arbritrage [12] 23
8 Moving averages on a BTC/USDT 5m chart 25
9 Bollinger bands on a BTC/USDT 5m chart 26
10 MACD on a BTC/USDT 5m chart 28
11 RSI on a BTC/USDT 1h chart 29
12 Layers of a neural network 31
13 Price impact on HOLO addition to the Binance Exchange 32
14 Tether price chart 33
15 The LSTM model implemented in keras 45
16 The graphical user interface 49
17 Sentiment analysis of twitter tweets. Blue: Bitcoin price, Red: Negative sentiment, Green: Positive sentiment 51
18 Model loss as the number of epochs increase 53
19 Actual and predicted price on the training set 53
20 Actual and predicted price on the testing set 53
21 Google interest of the search term "bitcoin" vs the price of Bitcoin 54
22 Tether price chart 55

List of Tables

1 Tri-arbitrage opportunities within 24 hours of running Python script 50
2 Results of backtesting the technical analysis trading strategies on different time frames of the BTCUSDT market 51
3 Profit shown using sentiment analysis 52
4 Settings used to train the model 52
5 Results from back-testing google trend interest on various search terms . . 54

Contents

1 Introduction **9**

1.1 Introduction . . . 9

1.2 Aims and Objectives . . . 10

1.3 Deliverables . . . 10

1.4 Dissertation Structure . . . 10

2 Background **12**

2.1 Cryptocurrencies . . . 12

2.1.1 Bitcoin . . . 12

2.1.2 Altcoins . . . 12

2.1.3 Wallets . . . 13

2.2 Market . . . 13

2.2.1 Cycles . . . 13

2.2.2 Manipulation . . . 15

2.2.3 Volatility . . . 16

2.2.4 Exchanges . . . 17

2.3 Data . . . 17

2.3.1 Price . . . 17

2.3.2 Volume . . . 18

2.3.3 Order Book . . . 18

2.4 Trading . . . 18

2.4.1 Back-testing . . . 18

2.4.2 Candlestick Charts . . . 18

2.4.3 Fees . . . 20

2.4.4 Orders . . . 20

2.4.5 APIs . . . 21

3 Trading Strategies **22**

3.1 Arbitrage . . . 22

3.1.1 Arbitrage between exchanges . . . 22

3.1.2 Tri-Arbitrage . . . 23

3.2 Technical Analysis . . . 24

3.2.1 Moving Averages . . . 24

3.2.2 Bollinger Bands . . . 26

3.2.3 Moving Average Convergence Divergence . . . 27
3.2.4 Relative Strength Index . . . 28
3.3 Sentiment Analysis . . . 29
3.4 Machine Learning . . . 30
3.4.1 Long short-term Memory . . . 30
3.5 Google trends . . . 32
3.6 New exchange additions . . . 32
3.7 Tether . . . 33

4 Design and Implementation 34
4.1 Python . . . 34
4.1.1 Numpy . . . 34
4.1.2 Pandas . . . 34
4.2 Data collection . . . 34
4.2.1 Matplatlib . . . 34
4.2.2 MongoDB and Websockets . . . 34
4.2.3 Coinmarket API . . . 36
4.3 Arbitrage . . . 36
4.3.1 Exchange arbitrage . . . 36
4.3.2 Tri-Arbitrage . . . 37
4.4 Technical Analysis . . . 40
4.4.1 Moving Averages . . . 40
4.4.2 Bollinger Bands . . . 41
4.4.3 Moving Average Convergence Divergence . . . 41
4.4.4 Relative Strength Index . . . 42
4.5 Sentiment Analysis . . . 43
4.5.1 Long short-term memory . . . 44
4.6 Google trends . . . 46
4.7 New exchange additions . . . 47
4.8 Tether . . . 47
4.9 Graphical User Interface . . . 48

5 Experiments and Evaluation 50
5.1 Arbitrage . . . 50
5.2 Technical Analysis . . . 50
5.3 Sentiment Analysis . . . 51

5.4 Long short-term memory 52
5.5 Google trends 53
5.6 New exchange additions 54
5.7 Tether 54
5.8 Evaluation 55

6 Conclusion **56**
6.1 Summary 56
6.2 Future work 56

Word Count: 9144

Declaration

No portion of the work referred to in this dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

1. The author of this dissertation (including any appendices and/or schedules to this dissertation) owns certain copyright or related rights in it (the Copyright) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

2. Copies of this dissertation, either in full or in extracts and whether in hard or electronic copy, may be made only in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has entered into. This page must form part of any such copies made.

3. The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the Intellectual Property) and any reproductions of copyright works in the dissertation, for example graphs and tables (Reproductions), which may be described in this dissertation, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

4. Further information on the conditions under which disclosure, publication and commercialisation of this dissertation, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy, in any relevant Dissertation restriction declarations deposited in the University Library, and The University Librarys regulations.

1 Introduction

1.1 Introduction

Cryptocurrencies have gained substantial interest recently because of the rapid increase of the price of these virtual currencies. Bitcoin, the most popular cryptocurrency, was able to increase from $7000 to $20000 during the winter of 2017. During this wave, an influx of new investors has joined which can be shown by the media reporting heavily during the price surge.

Due to the increase in interest, there has been more competition for making a profit by automatic trading. Automatic trading can be defined as computer programs that execute trades based on rules and conditions. It can be far more profitable to trade automatically than relying on manual trading as computer programs can execute instructions far superior than humans. The rules and conditions that are used by trading programs are generally based on defined trading indicators, market trends, and patterns.

The cryptocurrency market is exposed to market manipulation because of limited regulation, it makes finding an optimal trading strategy very hard because of random price movement. There can be a sudden flash crash which can be caused by exchanges getting hacked, pump and dump, liquidations and much more. This volatile nature of cryptocurrencies can make traditional trading strategies impractical.

There has been existing work on trading bots which have trading strategies implemented with them. The trading bots work by risking an initial amount of cryptocurrency and then trade based on trading strategies the user have initialized and then make trades to then make a profit. From the performance of these trading bots, it is shown that making a profit is not certain and is quite unreliable.

The best trading bots which perform well will be the ones that haven't been revealed to the public. This is to protect the trading strategy from being exposed to other people. In a scenario where there are multiple trading strategies that are similar, there will be competition and will result in which trading strategy performs the best. Therefore trading strategies are kept secret by nature - to avoid being exposed.

1.2 Aims and Objectives

The aim of the project is to explore different trading strategies and apply them into the cryptocurrency market to make a profit. In this thesis, I will be exploring different trading strategies, implementing the strategies and showing the results of their performance by performing back-testing.

In order to complete the aims, the following objectives have been identified:

- Identify and understand different trading strategies
- Implement the trading strategies
- Identify and analyze the performance between the trading strategies

1.3 Deliverables

The project will mainly focus on the implementation and results of the trading strategies. The trading strategies will be explained theoretically and implemented in code. Experiments will be conducted by back-testing and the results will be discussed and shown.

The project will also involve the development of a graphical user interface that will be used to execute trading strategies on historical data. This will essentially produce the same results from experiments that have been conducted.

1.4 Dissertation Structure

The dissertation is split into the following six chapters.

- Chapter 1 - Introduction: This chapter highlights the motivation for undertaking the project and also highlighting the aims and objectives associated with the project.
- Chapter 2 - Background: This chapter explains the basic knowledge of cryptocurrencies and other topics related to it. The knowledge will help in understanding later chapters of trading strategies and implementation details.
- Chapter 3 - Trading strategies: This chapter contains explanation of the trading strategies used in this project in a theoretical level.

- Chapter 4 - Implementation: This chapter will explain how the trading strategies mentioned in chapter 3 are implemented in code. This chapter also highlights any other code written which is used to aid in the implementations.
- Chapter 5 - Results: This chapter will show the results obtained from back-testing the trading strategies and present any notable discoveries.
- Chapter 6 - Conclusion: This chapter will provide an overall summary of the project, taking the results from chapter 5 into consideration. This chapter will also include mentions of future work and improvements.

2 Background

This chapter provides a basic introduction to cryptocurrencies.

2.1 Cryptocurrencies

Cryptocurrencies are digital currencies which are used as a medium of exchange. Cryptocurrencies are known to be highly risky investments which makes them speculative assets. Cryptocurrencies operate in a low regulation environment where there are not many rules in which traders are obligated to follow. This makes cryptocurrencies very volatile and exposed to market manipulation.

What makes cryptocurrencies special is that they are often decentralized and secure. Cryptocurrencies are decentralized with the use of blockchain. The blockchain is a distributed ledger which records transactions in blocks. These blocks are added to the blockchain by miners who use computing power to solve cryptographic problems. The blockchain can be seen as a decentralized database.

2.1.1 Bitcoin

Bitcoin was the first cryptocurrency which was invented by Satoshi Nakamato in 2009. It is the most popular cryptocurrency which most people are aware of. Bitcoin is designed to use as a digital currency where no central party has control over it, hence it is decentralized. At the time of writing, Bitcoin market capitalization is of 53% out of the whole cryptocurrency market.

Since Bitcoin is the most dominant cryptocurrency, it usually controls the flow of the market. Most cryptocurrencies are paired with Bitcoin, which means that to buy a certain cryptocurrency you must first buy Bitcoin and then trade it for that cryptocurrency. This, therefore, causes cryptocurrencies price to be impacted by the price of Bitcoin. When Bitcoin's price is increased, other cryptocurrencies' price also increases.

2.1.2 Altcoins

There are many cryptocurrencies that exist and have a specific goal or problem in which it seeks to resolve. A cryptocurrency which is not Bitcoin is referred to as an 'Altcoin'. Many altcoins are very different from Bitcoin. Some Altcoins use completely unique implementations whereas some are forks or clones of Bitcoin.

Cryptocurrencies aim to solve different types of problem. Etheruem, which is another popular cryptocurrency is a platform for developers to create decentralized web applications using smart contracts. Nano is designed to be a digital currency, just like Bitcoin, but has much more advantageous features such as instant transfer, no fees and is much more environmentally friendly. Binance Coin is a cryptocurrency which was created by the Binance Exchange to be used in their exchange to lower trading fees. There are many other cryptocurrencies and new cryptocurrencies are still being introduced which are tackling a wide range of fields.

2.1.3 Wallets

A cryptocurrency wallet is a software program that is responsible for holding cryptocurrencies such as Bitcoin. Most wallets will only support certain cryptocurrencies. There are different types of wallets ranging from hardware, paper and hosted. The wallet stores the private key and a corresponding public key is used to retrieve and send the actual cryptocurrencies. Therefore to own a wallet, one must know the private key.

2.2 Market

The cryptocurrency market is quite different from traditional markets. The cryptocurrency market is open 24/7, therefore, cryptocurrencies can be traded anytime. The market moves very fast and wild swings can happen any time which makes it very volatile than other markets. The market also has low regulation making it much riskier to trade on.

2.2.1 Cycles

The historical price movement of the cryptocurrency market has shown that the market moves in cycles. The cycle can be described in these steps[3]:

1. Cryptocurrency is accumulated at low prices
2. Price increases slowly
3. Price increases rapidly
4. Price goes down

5. Price crashes down harder

6. Repeat

Figure 1: Bitcoin price chart in 2017/2018

From figure 1, the price increase from April 2017 to December 2017 shows that the market was in a bull market. A bull market is when there is a sustained period of a price increase. After the price crash in January 2018, the market is seen to be in a bear market where prices are decreasing and are expected to decrease.

During the start of a new market cycle, accumulation occurs during a low point of the bear market. Cryptocurrencies are accumulated when they have been significantly had lost value from their all-time high prices. When everyone thinks that the bottom of the bear market is there, people will buy at these cheap prices and eventually will lead to a slow price increase. As the price slowly increases upwards, interest is gained in the market thus leading to more buying and price increase. This is when the bull market starts. When the price nears the previous all-time highs, the price usually increases harder as people do not want to miss out on making a profit. Eventually, the price will drop from people taking their profits, which will lead to a lot of selling. This will result back into a bear market and the cycle repeats.

2.2.2 Manipulation

Market manipulation occurs a large amount in the cryptocurrency market. Due to the deregulated nature of cryptocurrencies, market manipulation can occur without consequences.

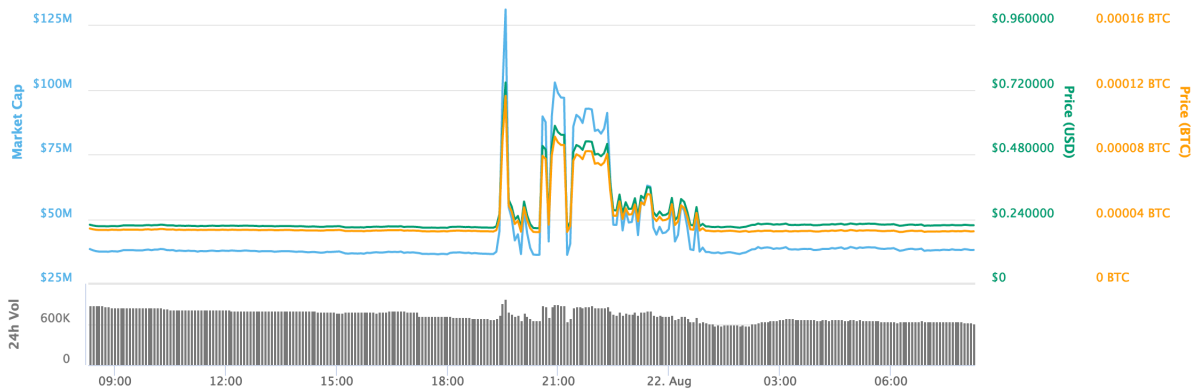

Figure 2: Pump and dump of a low market cryptocurrency

There are many different types of market manipulation tactics that happen in the cryptocurrency market. Pump and dump is a popular technique that happens often. Pump and Dump is when a large investor or a group aims to buy large amounts of a cryptocurrency at the same time to raise the price drastically and then selling at the top of the price increase. An example of a pump and dump can be seen clearly in Figure 2. The price was stable and suddenly the price increases by a large amount and then went back down again within minutes.

Figure 3: An example of a sell wall to restrict price increase

Another common market manipulation technique is buy and sell walls. This is used when trading with an exchange. Buy and sell walls show the quantity of the orders that are placed at different price amounts. A large individual can create these walls to dictate the price movement of a cryptocurrency. A big wall can create an illusion that there are more buyers or sellers which could indicate what direction the price is going to go. These walls can be removed at any time if the individual removes their orders, therefore, it is a prime tactic to control the price.

2.2.3 Volatility

There are many factors which play into that make cryptocurrencies very volatile. To some people, what makes the cryptocurrency market attractive is because of the volatility as this volatility is not that strong in other markets. Therefore there is potential for people to make a substantial amount of money in a short and long term.

One of the main reasons for this volatility is because the cryptocurrency market is mostly vaporware. Most of the cryptocurrencies' developers are still currently working on their product or solution to the problem they are trying to solve. Therefore holding cryptocurrency has no intrinsic value. Without any fundamental information, it can be hard to justify if a cryptocurrency is actually worth its price.

2.2.4 Exchanges

A cryptocurrency exchange is a platform for people to buy or trade cryptocurrencies. There are different types of exchanges available. There are exchanges where users can trade between fiat and cryptocurrencies. There are also exchanges for trading cryptocurrencies with different cryptocurrencies. Cryptocurrency exchanges can also be decentralized where the exchange is coded to interact with cryptocurrencies which are used to trade other cryptocurrencies or it can be completely centralized where the exchange fully owns the cryptocurrencies which are traded and only provides a front-end for users to trade with.

Cryptocurrency exchanges have been susceptible to getting hacked multiple times. One of the largest hacks happened in 2014. Mt. Gox was one of the most popular exchange for trading Bitcoin at the time. The exchange was handling 70% of volume of all bitcoin trading worldwide. The hack was revealed to be a software bug which allowed the hacker to change the price of bitcoin in the exchange and then led to the withdrawal of thousands of Bitcoin. The exchange lost $500 million worth of Bitcoin at the time. The result of the hack impacted Bitcoin's price the following days where it lost over 35% of it's price.

2.3 Data

Financial markets have been running for a long time and this results in enormous of data available related to trading. In the cryptocurrency market, there is new data coming every second as the market operates 24/7. This data is used strategically to compute many different trading indicators which are used to find patterns or trends that aid traders in making a beneficial trade.

2.3.1 Price

The most important piece of data in the cryptocurrency market is the price. The price represents the cryptocurrency's worth and this is what most traders are looking at to influence their trades. Price is, therefore, the most important factor when trading cryptocurrencies and it is used heavily in trading strategies.

2.3.2 Volume

There are other variables which are also important when making trades or also have an influence that affects traders' decisions. Volume is another measurement that can be useful to take account when trading. Volume is usually measured by the total sum of the value of every trade that has happened in a day.

2.3.3 Order Book

The order book also provides a lot of information that can be useful for traders. The order book contains a stacked list of trades that users have made that are not fulfilled. A trade shows the price a user is buying or asking and also the amount of quantity for a cryptocurrency. The order book is available to look at on the platform an exchange provides.

2.4 Trading

2.4.1 Back-testing

Back-testing is the process of executing trading strategies on historical data. This is useful as back-testing can be used to measure the performance of trading strategies. Trading strategies that have worked in the past are more likely to work in the future as well. Trading strategies are often back-tested and altered to find an optimal trading strategy.

2.4.2 Candlestick Charts

The data that is available in the cryptocurrency market is huge and to interpret this data in its raw form is near impossible. Therefore traders will use different types of charts to help visualize this data.

There are different types of charts that are used in the cryptocurrency market. One of the major charts that are used is the candlestick chart. The candlestick chart represents the price of an asset in a certain time interval. An individual candlestick in the chart contains price data for a single time instance. The candle represents four main price indications:

- Open - This is the price of when the time interval has started.
- Close - This is the price of when the time interval has closed.

- High - This is the highest price that was achieved during the interval.
- Low - This is the highest price that was achieved during the interval.

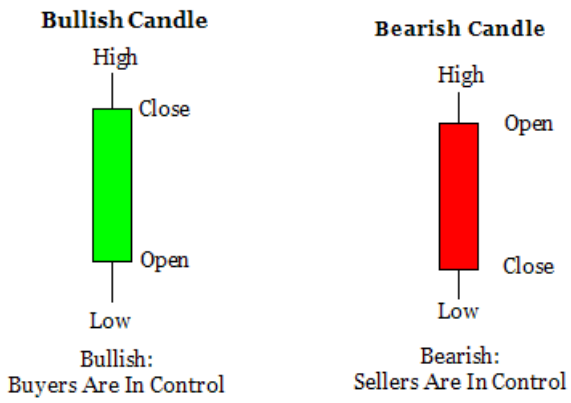

Figure 4: Characteristics of candlesticks

There are two different types of candlesticks in a candlestick chart. There is a bearish candle and a bullish candle. A bearish candle is when the closing price is greater than the opening price of the candle. Conversely, if the opening price is greater than the closing price, the candle is viewed as bullish. Bullish candles give the signal of the price going up because the price has gone up during the time interval and vice versa for a bearish candle. A red color is given to the candle to indicate a bearish candle and a green color is given for a bullish candle.

(a) 45 Minute (b) 1 Day

Figure 5: Latest candlestick charts of Bitcoin taken at 5th September 2018

Candlestick charts can be viewed in different time intervals. Using different time intervals can reveal different information to the trader. In Figure 5, the two different

price charts show different price direction the price of Bitcoin is heading. In the 45m chart, the price is seen to be increasing whereas, in the 1-day chart, the price is seen to be decreasing. This, therefore, shows that in the short-term, the price is in an uptrend and in the long-term, the price is in a downtrend. Traders, therefore, look at different time intervals to better understand the market direction.

2.4.3 Fees

When a cryptocurrency is being transferred from one wallet to another, a transfer fee is paid to the miners to keep the network alive. Not all cryptocurrencies have fees and some block-chain networks do not use the model of having miners to hold an incentive. However, for most cryptocurrencies, there will be a fee.

There is also trading fees that happen when a trader makes a trade in an exchange. The trading fee can vary between exchanges. Trading fees can also be different depending on whether an order is a maker or taker. Taker fees are applied when an order immediately matches another order that is in the order book whereas a maker fee is applied when an order does not immediately match and it, therefore, stays in the order book [7]. Usually, a maker fee is much less than the taker fee because it provides liquidity to the order book by creating a new order.

2.4.4 Orders

There are many different types of orders that a trader can make to aid in their trading. Making the use out of the tools available will help the trader in making a profit in the long-term. The types of orders you can make on an exchange are [11]:

- Market Order - The order is placed and executed immediately at the best market price.
- Limit Order - The order is specified a certain price to buy or sell. The order can be immediately be fulfilled or it can be left in the order book.
- Stop Order - The order is made when the price of a cryptocurrency has reached a specific price.
- Stop Limit Order - When the price specified has reached, a limit order is placed.

2.4.5 APIs

Exchanges provide application programming interfaces (APIs) to interact with the exchange by code. This enables traders to make trades programmatically. APIs that are provided usually have a rate limit. A rate limit will limit the number of requests that can be made to the APIs. Rate limits will vary between exchanges. The APIs provided by the exchange typically provide many endpoints which can include:

- Checking balance of cryptocurrencies
- Withdrawing/Depositing cryptocurrencies
- Showing historical/live data
- Creating orders

3 Trading Strategies

A trading strategy is the process of making trades that follow predefined rules.

3.1 Arbitrage

An arbitrage opportunity occurs when the price difference between two markets that buy or sell the same product. A trader can buy a cryptocurrency in one market and then sell it in another market for profit. In the cryptocurrency market, a market, for example, Bitcoin and the U.S Dollar, appears in many exchanges and therefore there is a chance for many arbitrage opportunities.

Arbitrage opportunities vanish very quickly as most computer programs are taking advantaging of this. It is near impossible for traders to make a profit by arbitrage by manually trading. Automated trading with the use of APIs is far more successful than manual trading as the speed of computer programs to analyze and execute trades far exceed human capabilities.

Arbitrage trading compared to other trading strategies, in theory, is risk-free. This is because if the prices of a cryptocurrency are known in two different exchanges, the profit can be calculated by buying and selling in these exchanges. Also taking into account the trading and transferring fees, a computer program could be created to calculate a possible profit and only execute trades when there is profit.

3.1.1 Arbitrage between exchanges

The same markets are found in multiple exchanges. In these markets, the price is kept near one another because of arbitrage. If there is a large price difference, traders will take advantage of arbitrage.

Figure 6: Prices between different markets of NANO/BTC

In Figure 6, the price difference can be shown of different exchanges for a market. There is quite a short price discrepancy but a small profit can still be made. The process of making a profit by arbitrage trading can be defined below:

1. Find the latest bid and ask prices of different markets
2. Calculate if profit can be made (taking account of fees)
3. If profit can be made, Buy cryptocurrency in an exchange
4. Transfer the cryptocurrency in another exchange
5. Sell the cryptocurrency in the exchange for a profit

3.1.2 Tri-Arbitrage

Tri-arbitrage is the process of making a profit in three different trades by taking advantage of the price difference. It has numerous advantages over arbitrage between exchanges. Tri-arbitrage makes use of different markets compared to the same markets so, therefore, it is possible to utilize this in one exchange. Another advantage is that tri-arbitrage does not require the need of transferring cryptocurrencies.

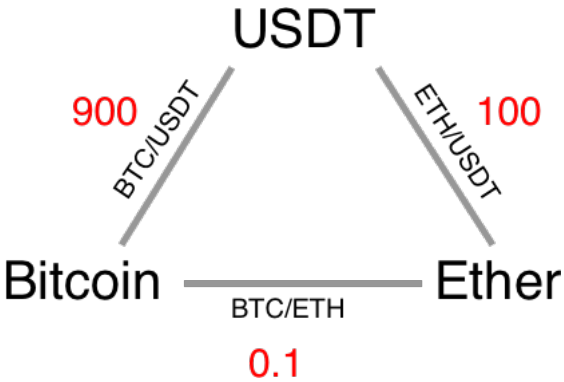

Figure 7: Example of tri-arbitrage [12]

In figure 7, the markets for Bitcoin, USDT, and Etheruem are shown and their prices. A profit can be made by making use of tri-arbitrage by making the following trades:

- Trade 1 Bitcoin for 10 Etheruem
- Trade 10 Etheruem for 1000 USDT
- Trade 1000 USDT for 1.1 Bitcoin

3.2 Technical Analysis

Technical analysis is the study of finding market trends and patterns with the use of historical data. Technical analysis is a traditional method that has been used in many financial markets. Technical analysis help traders make decisions on trades as it helps them understand the market better than basic price charts.

3.2.1 Moving Averages

One of the most basic techniques related to technical analysis is moving averages. Using a moving average can be beneficial to look at as it provides a smoothened view of which direction the price is heading.

The simple moving average (SMA) is the running moving average of the price over a certain period. It is the easiest out of the other moving averages to calculate. In the equation above, the simple moving calculation is shown to be the sum of all the latest prices and dividing by the number of prices given by n. This is simply the average of the list of prices.

$$SMA_i(n) = \frac{1}{n}\sum_{i=1}^{n} p_i \tag{1}$$

Calculating the simple moving average over a period will produce a list of values which can be plotted as a line chart. Figure 8 shows an example of a simple moving average using a rolling window of 5 prices. Using a different window length can produce different moving averages.

The weighted moving average (WMA) is similar to the simple moving average with a few changes. More priority is given to the latest prices with the use of weights. Weights are values which are scaled to the prices in the order they appear. The latest price will have the highest weight and the oldest price will have the lowest weight.

$$WMA_i(n) = p_i w_i \tag{2}$$

To calculate the weighted moving average of a price point, the sum of each price point within the rolling window is multiplied by a weight w. Weightings are given by using arbitrary numbers.

The exponential moving average (EMA) is another moving average that prioritizes re-

cent prices than old prices. Instead of using a consistent decrease such as the weighted moving average, the weighting decreases exponentially.

$$K = \frac{2}{n+1} \tag{3}$$

$$EMA_i(n) = (p_i - EMA_{i-1}) \times K + EMA_{i-1} \tag{4}$$

To calculate the exponential moving average, first, a value K needs to be calculated which acts as a multiplier. It is calculated by dividing 2 by n + 1 where n is the window length. Then the exponential moving average can be calculated by getting the current price point and subtracting the value by the previous price point's EMA. This value is then multiplied by k and the previous price point's EMA as described by the equation.

Figure 8: Moving averages on a BTC/USDT 5m chart

In Figure 8, the moving averages plotted in a price chart is shown. From the chart, it can be seen that the WMA and EMA respond to a large price change faster than the SMA, therefore, the SMA is less sensitive to price changes. Moving averages can be used to predict what direction the price is heading. An uptrend will indicate a bullish signal whereas a downtrend will indicate a bearish signal. Therefore a trading strategy can be formed which makes use of moving averages where the trades should be made at when the direction of the moving average changes. When the moving average change turns from positive to negative, the algorithm should send a sell order as the moving average would indicate the price to go down and vice versa.

3.2.2 Bollinger Bands

Bollinger bands is a trading indicator that is used to track volatility. The Bollinger bands use the simple moving average. A simple moving average is plotted and then two new lines are drawn to indicate the upper band and lower band of the simple moving average. These bands are calculated by finding the difference of two standard deviations away from the simple moving average. Standard deviation is used as it is a good measurement for volatility.

$$MiddleBand = SMA \tag{5}$$

$$UpperBand = SMA + (2 \times \sigma(p) \tag{6}$$

$$LowerBand = SMA - (2 \times \sigma(p) \tag{7}$$

To calculate the Bollinger bands, first, the simple moving average needs to be calculated. Then the standard deviation needs to be found of the price points. The upper and lower bands can be then found by adding and subtracting the standard deviation by the simple moving average.

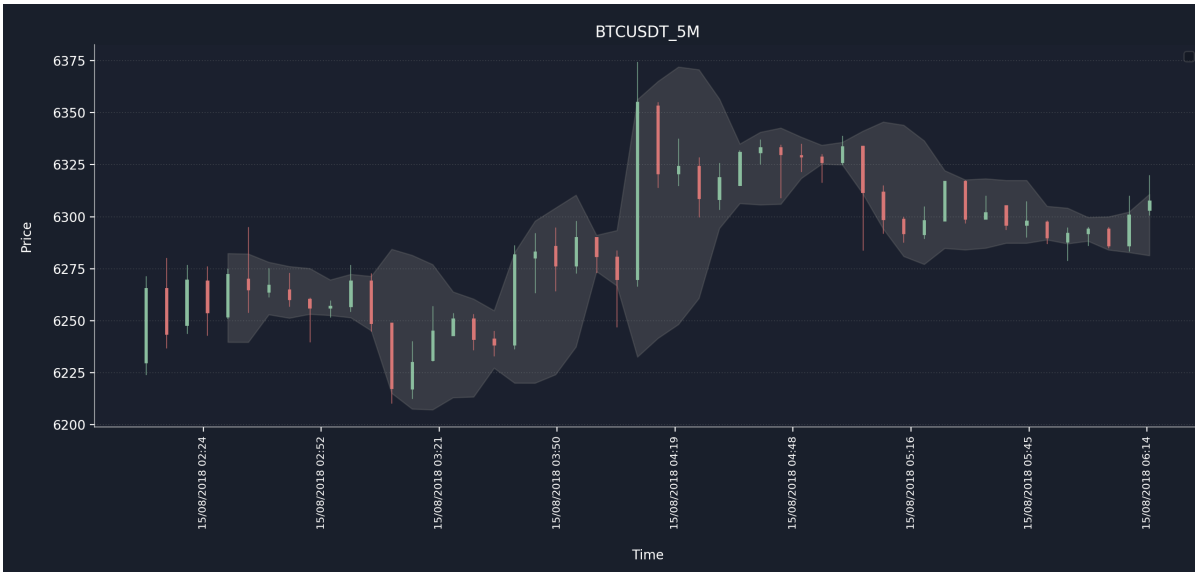

Figure 9: Bollinger bands on a BTC/USDT 5m chart

In Figure 9, the upper and lower bands are plotted. The idea of Bollinger bands is that when the bands are very narrow because of low volatility, a large price movement is

expected to follow. When the candles touch the bands, the price is seen to be over-brought or over-sold depending on the which band is touched. A trading strategy using Bollinger bands can be created and would be defined by triggering buy or sell orders depending if the bands are touching the candles.

3.2.3 Moving Average Convergence Divergence

The moving average convergence divergence (MACD) is an indicator for finding market momentum and direction. The MACD is shown using three components which are the MACD line, signal line, and the histogram. The crossovers between the MACD line and signal line provide signals to uptrends and downtrends. The histogram helps with showing how strong a momentum is.

$$FastEMA = EMA(12, Prices) \tag{8}$$

$$SlowEMA = EMA(26, Prices) \tag{9}$$

$$MACD = FastEMA - SlowEMA \tag{10}$$

$$Signal = EMA(9, MACD) \tag{11}$$

$$Histogram = MACD - Signal \tag{12}$$

First, there is the MACD line which is the difference between the fast EMA line and the slow EMA line. Then the signal line is an EMA of the MACD line. Lastly, the histogram is the difference between the MACD and the signal. The parameters for the MACD are usually 12 for the fast EMA, 26 for the slow EMA and 9 for the signal.

Figure 10: MACD on a BTC/USDT 5m chart

The MACD shows that whenever the MACD line crosses over the signal line, there is a bullish signal and shows an uptrend. Conversely, if the MACD line crosses down the signal line, then there is a bearish signal and shows a downtrend. The histogram shows this clearly as when there is a crossover in between the MACD line and the signal line, the histogram changes direction. A trading strategy could be created to utilize the histogram in making decisions when the value crosses 0.

3.2.4 Relative Strength Index

The relative strength index is a simple indicator which uses prices within a timeframe to calculate if an asset is oversold or over-brought. It is a useful indicator as it measures how strong a recent price action has been. It is usually used within long time-frames as a short time-frames result in a lot of noise which makes the indicator not very accurate.

$$RS = \frac{AverageGain}{AverageLoss} \tag{13}$$

$$RSI = 100 - \frac{100}{1 + RS} \tag{14}$$

The RSI can be calculated by finding the average gain and loss within a time-frame specified. Then the relative strength can be identified with the average gain divided by the average loss. The RSI can, therefore, be calculated by subtracting 100 from 100 divided by 1 + the relative strength as shown in the formula above.

Figure 11: RSI on a BTC/USDT 1h chart

When the RSI is below 30, the asset is then said to be oversold. Conversely, when the RSI is above 70, the asset is then said to over-brought. A trading strategy can be derived by using these conditions to trigger buy or sell signals.

3.3 Sentiment Analysis

Sentiment analysis is the study of categorizing opinions or views expressed by either positive, negative or neutral. To categorize a sentence or paragraph into positive or negative, the phrases are sent to the algorithm in which it returns the category. This algorithm is possible with the use of machine learning.

The Naive Bayes classifier is a well-known solution to text classification problems. The Naive Bayes classifier is a probabilistic model related to the Bayes Theorem. The classifier makes an assumption that features are independent of one another despite the given class. This gives the classifier a high bias which results in strong predictions.

The classifier is trained with negative or positive text documents. This allows the classifier to know how many times a word appears in a positive or negative document in which it uses this to calculate the probabilities of it belonging it a certain class.

$$y \leftarrow \textbf{argmax}\ P(y = y_k) \prod_i P(x_i \mid y = y_k) \tag{15}$$

The classification rule for Gaussian Bayes classifier is defined above. Given a word x,

the classifier calculates the probability for each given class and chooses the class which has given the largest probability.

A trading strategy can be created to use sentiment in certain sentences using sentiment analysis. The conditions to trigger buy or sell signals will depend on the sentiment. If the positive sentiment is larger than the negative sentiment, it would indicate a buy signal as positive sentiment will correlate to a price increase and vice versa.

3.4 Machine Learning

Machine learning is an exploding field currently in financial markets. Many institutional organizations use machine learning to generate predictive models. These models are used in several areas including fraud detection, algorithmic trading, portfolio management and much more [13].

3.4.1 Long short-term Memory

Machine learning models are useful for categorization problems where a model is able to categorize unseen data. However, in financial markets, this is not useful is it is not able to predict values. With the introduction deep learning in recent years, neural networks have been key in recognizing patterns.

Neural networks is simply a network of neurons which are used to transfer information. A neuron is responsible for receiving input, transforming the input to some mathematical calculation, and then outputting the data to possible output elements. Each neuron is assigned a weight vector and the weights can be configured during the training phase. When data is sent to the neuron, the weighted sum is calculated and sent to possible outputs. Neurons can also be activated or deactivated depending on the network.

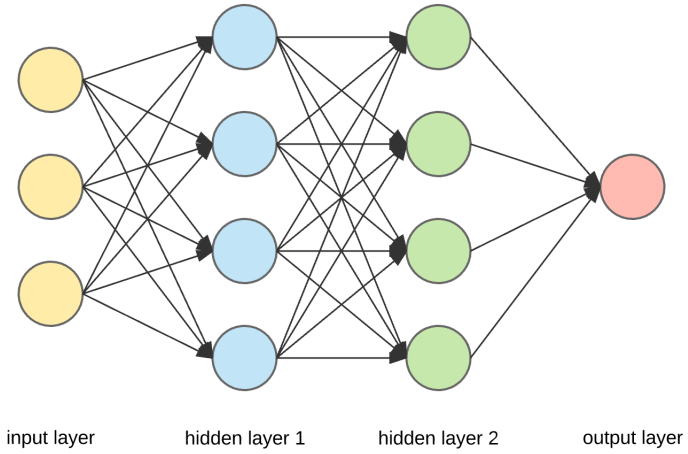

Figure 12: Layers of a neural network

In neural networks, there are layers which contain a list of neurons at a specific depth in the network. The data within a network is passed through these layers. A neural network typically contains an input layer, hidden layers, and an output layer. Input data is fed into the input layer, which then activates some nodes and then affects the weights on the neurons of the next layer. This process is repeated until the output layer, where the network makes a prediction.

Recurrent neural networks (RNN) are a special type of neural networks that have one extension to normal neural networks. Recurrent neural networks contain edges where the network does not have to go in one direction. Recurrent neural networks can loop back to previous neurons which in turn allows the network to keep track of previous states.

Long short-term memory (LTSM) is a recurrent neural network that solves the vanishing gradient problem [15]. The problem happens in the training phase related to neural networks using gradient optimization techniques. The problem is that the neurons learn more slowly in early layers than in later layers. LTSMs use cells to capture the memory. The cells are used in conjunction with previous states and inputs to capture sequences.

LTSMs can be useful with time series data as it can make predictions on future prices. A trading strategy can be created by making buy orders if the classifier predicts the price to go up and by making a sell order if the classifier predicts that the price will go down.

3.5 Google trends

Google is the most popular search engine in the world. By finding data on when a search term was the most popular throughout recent years, interest can be measured. Interest in cryptocurrency is strongly correlated with price. When the price rapidly increases, more people are known about cryptocurrencies and they are on the subject of missing out on the price action. In the cryptocurrency market, herd mentality is very much true and this can result in a fast price movement.

By finding interest in using basic cryptocurrency search terms using Google, a trading strategy can be formed by using this data. When interest is high, a buy order is executed and when interest decreases, a sell order is executed.

3.6 New exchange additions

Cryptocurrencies are often added to existing exchanges so more traders can buy new cryptocurrencies. Exchanges which are popular have a large amount of trading volume. When a cryptocurrency gets listed in the exchange, this usually results in the price surge as more people will be able to buy the newly added cryptocurrency.

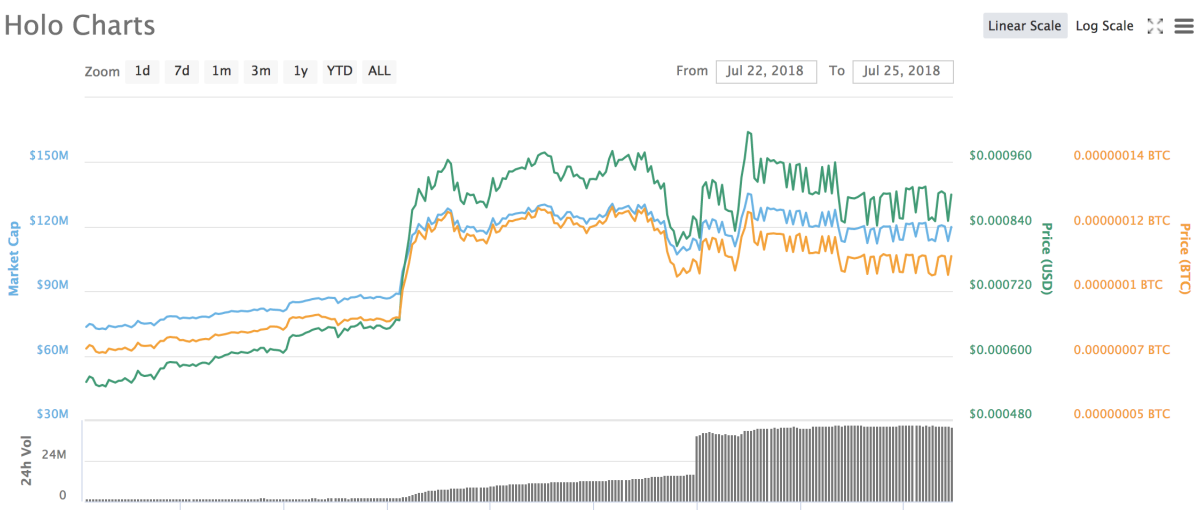

Figure 13: Price impact on HOLO addition to the Binance Exchange

In the figure above HOLO'price increased by 30% because the cryptocurrency was added to the Binance exchange . A trading strategy, therefore, can be created to automatically create buy orders when a cryptocurrency has just been added to an exchange.

3.7 Tether

Tether (USDT) is a special cryptocurrency that was created to be a stable coin. A stable coin is a cryptocurrency in which the price is pegged to something and therefore does not change in rapid price movements. Tether is pegged to the U.S dollar and therefore the price is always near one dollar. However in high volatility phases in the cryptocurrency market, tether's value can change and this gives data about the state of the market.

Figure 14: Tether price chart

Tether is used as a fiat alternative in the cryptocurrency market. When a trader wishes to hold something that does not change in value, Tether is used for this. A scenario for using tether is during a price downtrend, if Tether is brought at the start of a downtrend and then sold at the rise of an uptrend, profit can be made as a trader's cryptocurrency would have gone up. However, in sudden price surges or crashes, tether's price can change where it is not pegged to $1. Usually, the price is a bit higher when the market crashes and the price is a bit shorter when the market is rising rapidly. A trading strategy can involve creating buy or sell orders when tether's price changes and it is not $1.

4 Design and Implementation

This section describes the implementation of the trading strategies in detail and also highlighting any design choices that were made.

4.1 Python

The programming language chosen to implement the trading strategies is Python. Python is a simple language to quickly start up working with and it makes it easy to read. Many packages are available in Python that makes it simple to work with financial data.

Each trading strategy is written in a single Python script. This makes it easier to manage and it allows it to be easily used in other scripts or systems.

4.1.1 Numpy

The `numpy` package in Python is used to implement the trading strategies. The package provides a high-level API for manipulating arrays and providing a set of useful functions.

4.1.2 Pandas

The `pandas` package provides many useful data structures to use with financial data. It is a key tool used in data analysis.

4.2 Data collection

4.2.1 Matplatlib

The `matplotlib` package was used to plot graphs in Python. It provides a numerous of visualization charts that is used to better show data that is gathered. The library works together with `numpy` and `pandas`

4.2.2 MongoDB and Websockets

To back-test the trading strategies, price data was needed for cryptocurrencies. To collect data to back-test, there were several ways to get the data. The data that is used mostly with the trading strategies are candlestick data.

A database was needed to store data that was going to be collected. MongoDB was

used instead of a relational database because of the complex structure that financial data is in. MongoDB also makes it easy to start working with documents because they are schemeless. Querying is also supported and this makes searching for data simple.

Websockets allow an open and persistent connection between a client and a server. Websockets was used to collect candlestick data for different markets and timeframes. The Binance exchange provides a websocket stream for candlestick data. The `python-binance` package in Python provides several endpoints to interact with the Binance API. A Python script was therefore created to collect data from the streams and add the data to the MongoDB database.

The `start_multiplex_socket` method is used from the package to create a multiple web-socket connection with the streams shown in the code. This allows me to collect candlestick information for the following markets in the 1m, 5m, 30m, 1h, 4h and 1d timeframes:

- BTC/USDT
- TRX/ETH
- NANO/BTC
- NEO/BNB
- ZIL/BTC

The Python script will constantly be collecting data as soon as data is sent to the streams and therefore the script needs to be running constantly. A solution to this problem is for the script to run on a cloud server. The database also resides in the cloud server and the script will continuously save to the database.

For backtesting, the data in the MongoDB database needs to be sent to another MongoDB instance to run scripts locally. Therefore the following shell script dumps the database from the server to a local MongoDB instance:

```
#! /bin/bash
mongodump -h 159.65.230.56 --port 27017 -d crypto
```

```
wait
mongorestore -d crypto dump/crypto/
wait
rm -r dump
```

4.2.3 Coinmarket API

The Coinmarket API provides historical data on multiple cryptocurrencies which makes it ideal for long-term back-testing. Historical price data for cryptocurrencies is available since 2013.

```python
def get_cryptocurrency_data(full_name, short_name=True):
  market_data = pd.read_html("https://coinmarketcap.com/currencies/" +
      full_name +
      "/historical-data/?start=20130428&end="+time.strftime("%Y%m%d"),
      flavor='html5lib')[0]
  market_data = market_data.assign(Date=pd.to_datetime(market_data['Date']))
  market_data['Volume'] = (pd.to_numeric(market_data['Volume'],
      errors='coerce').fillna(0))
  if short_name:
    market_data.columns = [market_data.columns[0]] + [short_name + '_' + i for
        i in market_data.columns[1:]]
  return market_data
```

Code 1: Method to get historical price data using the Coinmarket API

The Python function above is used in several Python scripts to get cryptocurrency historical data easily. The name of the cryptocurrency needs to be passed as a parameter and then a URL is formed and made a request call the Coinmarket API. The historical price data is then returned.

4.3 Arbitrage

4.3.1 Exchange arbitrage

To implement an algorithm which takes advantage of exchange arbitrage, the order book of exchanges needed to be scanned to find the latest ask and buy orders. The `ccxt` package was used to interact with numerous cryptocurrency exchanges easily. The trading strategy works by continuously scanning the same markets from two exchanges, then the

order books are scanned and arbitrage opportunities are found. Buy and sell orders are only placed if profit can be made. There is a delay made between the continuous calls to each market to stay within the limits of the rate limits imposed by the exchanges.

```python

for symbol in symbols:
        try:
            b_price = binance.fetch_order_book(symbol)['asks'][0][0]
            k_price = kucoin.fetch_order_book(symbol)['bids'][0][0]
        except Exception:
            print('error in: ', symbol)
        finally:
            b_btc = 0.01
            b_alt = 0

            k_btc = 0

            b_btc = b_btc - (b_btc * 0.001) #trading fee

            b_alt = b_btc/b_price
            k_alt = b_alt

            k_alt = k_alt - (k_alt * 0.001) #trading fee

            k_btc = k_alt*k_price

            if k_btc > b_btc:
                print(k_btc, ' - ', symbol)
                #PROFIT CAN BE MADE: BUY OR SELL HERE
            else:
                print('fail: ', symbol)
            time.sleep(delay)
```

Code 2: Trading strategy of exchange arbitrage

4.3.2 Tri-Arbitrage

Since tri-arbitrage occurs within the same exchange, websockets can be used to get constant data on order books. This is more popular than making requests using APIs as it

is far more instant and there are no rate limits for using websockets.

The `python-binance` provides websockets to get live order book data. The `get_depth_cache` endpoint sets up a websocket stream to get order book data. Using this, finding tri-arbitrage opportunities becomes easy to find.

```

while True:
    time.sleep(0.5)

    print(count)
    start_btc = 0.0020
    btc = start_btc
    print("Start with ", btc, " BTC")

    altbtc_ask = dcm_altbtc.get_depth_cache().get_asks()[0]

    value = altbtc_ask[0] * altbtc_ask[1]
    if value > btc:
        alt_quantity_before_fee = btc / altbtc_ask[0]
        alt_quantity_before_fee = helper.round(alt_quantity_before_fee,
            altbtc_stepsize)
        btc = 0
        print("Traded ", start_btc," BTC for ", alt_quantity_before_fee, " ",
            alt_symbol)
    else:
        alt_quantity_before_fee = value / altbtc_ask[0]
        alt_quantity_before_fee = helper.round(alt_quantity_before_fee,
            altbtc_stepsize)
        btc -= value
        print("Traded ", value, " BTC for ", alt_quantity_before_fee, " ",
            alt_symbol)

    alt_quantity_after_fee = alt_quantity_before_fee -
        (alt_quantity_before_fee * fee)
    alt_quantity_after_fee = helper.round(alt_quantity_after_fee,
        altusdt_stepsize)

```

```
    altusdt_bid = dcm_altusdt.get_depth_cache().get_bids()[0]

    value = altusdt_bid[0] * altusdt_bid[1]

    if altusdt_bid[1] > alt_quantity_after_fee:

        usdt_quantity_before_fee = alt_quantity_after_fee * altusdt_bid[0]
        usdt_quantity_before_fee = helper.round(usdt_quantity_before_fee,
            altusdt_stepsize)
        print("Traded ", alt_quantity_after_fee, ' ', alt_symbol, " for ",
            usdt_quantity_before_fee, " USDT")

        usdt_quantity_after_fee = usdt_quantity_before_fee -
            (usdt_quantity_before_fee * fee)
        btcusdt_ask = dcm_btcusdt.get_depth_cache().get_asks()[0]

        value = btcusdt_ask[0] * btcusdt_ask[1]

        if value > usdt_quantity_after_fee:
            btc_quantity_before_fee = usdt_quantity_after_fee / btcusdt_ask[0]
            btc_quantity_before_fee = helper.round(btc_quantity_before_fee,
                btcusdt_stepsize)
            btc_quantity_after_fee = btc_quantity_before_fee -
                (btc_quantity_before_fee * fee)

            btc += btc_quantity_after_fee
            if btc > start_btc:
                count += 1

                #Create Order Here
            print(btc)
        else:
            print("Unable to trade USDT for ", (usdt_quantity_before_fee /
                btcusdt_ask[0]), " BTC")



    else:
```

```
        print("Unable to trade ", alt_quantity_before_fee, ' ', alt_symbol, "
            for ", altusdt_bid[1], " USDT")
```

Code 3: Trading strategy of tri-arbitrage

4.4 Technical Analysis

The calculations of the trading indicators in technical analysis are mathematical. A python package called `TA-Lib` is a Python wrapper for a popular technical analysis library. The package provides many methods for computing the trading indicators and is simple to use.

4.4.1 Moving Averages

The moving averages can be calculated by simply using the `SMA`, `WMA` and `EMA` methods provided by the package. The prices and window length need to be given as parameters.

The trading strategy using the moving averages works by first finding the current moving average and the previous moving average. The percent change between the previous and current moving average is found. If the change is greater than 2%, then a buy order is placed as it indicates a price increase. Conversely, a sell order is placed if the percent change is less than 2%.

The trading strategy is implemented in code below:

```
    for i in range(1,len(sma)):
        curr = sma[i]
        prev = sma[i - 1]
        change = ((float(curr) - prev) / prev) * 100

        if change > 0.02 and last_position == Type.SELL:
            print('BUY @', data[i]['time_str'])
            last_position = Type.BUY
            stack /= data[i]['close']

        if change < -0.02 and last_position == Type.BUY:
            print('SELL @', data[i]['time_str'])
            last_position = Type.SELL
```

```
            stack *= data[i]['close']
```

Code 4: Trading strategy of moving averages

4.4.2 Bollinger Bands

The Bollinger bands was calculated using the `BBANDS` method from the `TA-Lib` package. The method takes in the prices and a window length as parameters. The method returns the lower, middle and upper bands.

The trading strategy works by finding the lower and upper bands. If the current closing price is lower than the lower band, it indicates a buy signal as the cryptocurrency is said to be oversold. Conversely, if the closing price is higher than the upper band, a sell signal is placed. A factor variable also exists which influences the price when compared to the bands.

The trading strategy is implemented in code below:

```
 for i in range(1,len(lower_bands)):
       price = data[i]['close']
       factor = price * 0.001

       if price - factor < lower_bands[i] and last_position == Type.SELL:
           print('BUY @', data[i]['time_str'])
           last_position = Type.BUY
           stack /= data[i]['close']

       if price + factor > upper_bands[i] and last_position == Type.BUY:
           print('SELL @', data[i]['time_str'])
           last_position = Type.SELL
           stack *= data[i]['close']
```

Code 5: Trading strategy of bollinger bands

4.4.3 Moving Average Convergence Divergence

The MACD was calculated using the `MACD` method from the `TA-Lib` package. The lengths of the slow EMA, fast EMA and signal are given as parameter. The method returns the MACD line, signal line, and the histogram.

The histogram from the MACD calculation is used in the trading strategy. When the histogram value is greater than 0, there is an uptrend and therefore triggering a buy order. Conversely, when the value is less than 0, a sell order is placed.

The trading strategy is implemented in code below:

```
for i in range(1, len(histogram)):
  value = histogram[i]

  if value > 0 and last_position == Type.SELL:
      last_position = Type.BUY
      print('BUY @', data[i]['time_str'])
      stack /= data[i]['close']

  if value < 0 and last_position == Type.BUY:
      last_position = Type.SELL
      print('SELL @', data[i]['time_str'])
      stack *= data[i]['close']
```

Code 6: Trading strategy of moving average convergence divergence

4.4.4 Relative Strength Index

The relative strength index is calculated by using the `RSI` method from the `TA-Lib` package. The window length is given as a parameter. The RSI is returned from the method.

The trading strategy is created by placing a sell order when the RSI is said to be overbought at more than 70 and placing a buy order when the RSI is below 30 as it is said to be oversold.

```
for i in range(1, len(rsi)):

    if rsi[i] < 30 and last_position == Type.SELL:
        last_position = Type.BUY
        print('BUY @', data[i]['time_str'])
        stack /= data[i]['close']

    if rsi[i] > 70 and last_position == Type.BUY:
```

```
            last_position = Type.SELL
            print('SELL @', data[i]['time_str'])
            stack *= data[i]['close']
```

Code 7: Trading strategy of relative strength index

4.5 Sentiment Analysis

Twitter is a social media platform where users can send tweets which are just text messages for other users to see. The tweets come from users all around the world so therefore it can be a useful platform for collecting data to show sentimental value.

To analyze Twitter tweets, the `python-twitter` package is used. The package is a wrapper over the official Twitter API and provides an easier interface of interacting with the Twitter API. The `GetSearch` method from the API provides a way to get recent tweets by providing a search query as a parameter.

The `nltk` package is used for sentiment analysis. The `SentimentIntensityAnalyzer` class provided by the package is used for finding sentiment within a text piece. The class uses bigrams on the Naive Bayes and maximum entropy classifier for its implementation. The method `polarity_scores` takes in a string and returns the sentimental scores. The scores are between 0 and 1.

A trading strategy can be created by making use of the sentimental value. If there is a positive sentiment, then a buy order should be signaled. Conversely, if the overall sentiment is negative, a sell order should be placed. In the trading strategy below, a buy order is placed if the negative sentiment is large (greater than 0.5) a sell order is placed and a buy order is placed if the negative sentiment is small (lower than 0.5).

```
        for i in range(0, len(neg)):

         if neg[i] < 0.5 and last_position == Type.SELL:
            print('BUY @', times[i])
            last_position = Type.BUY
            stack /= prices[i]

```

```
        if neg[i] > 0.5 and last_position == Type.BUY:
            print('SELL @', times[i])
            last_position = Type.SELL
            stack *= prices[i]
```

Code 8: Trading strategy using sentimental value

4.5.1 Long short-term memory

There are numerous packages available in Python that work with machine learning. The `keras` package will be used to implement the LSTM network. Keras is a machine learning library which is specific for working with neural networks. It provides a high-level API to implement numerous neural networks.

The time series data that will be used to train and test the model, will be Bitcoin historical price data. The price data will be coming from the Coinmarket API. The task of the model will be to predict the next closing price given a sequence of previous prices.

The build the LSTM model, a series of steps are needed to create the model. In keras, models are built sequentially and therefore the following code builds the LSTM model:

```
model = Sequential()
model.add(Bidirectional(LSTM(neurons, return_sequences=True),
    input_shape=(inputs.shape[1], inputs.shape[2])))
model.add(Bidirectional(LSTM(neurons)))
model.add(Dense(1))
model.add(Activation('tanh'))
model.compile(loss='mse', optimizer='adam', metrics=['mae'])
model.summary()
return model
```

Code 9: LSTM model built on keras

The LSTM model is a sequential model. This means that the model will be built layer by layer in order. The first two layers are of type bidirectional LSTM. The first parameter of the LSTM takes in a number which represents the number of neurons the layer will have. The `return_sequences` boolean will return the full sequence as the output to the next layer. A Dense layer is then added which represents a layer which is fully connected which represents the output layer. Then a hyperbolic tangent activation is applied.

The model can be visualized in figure 15:

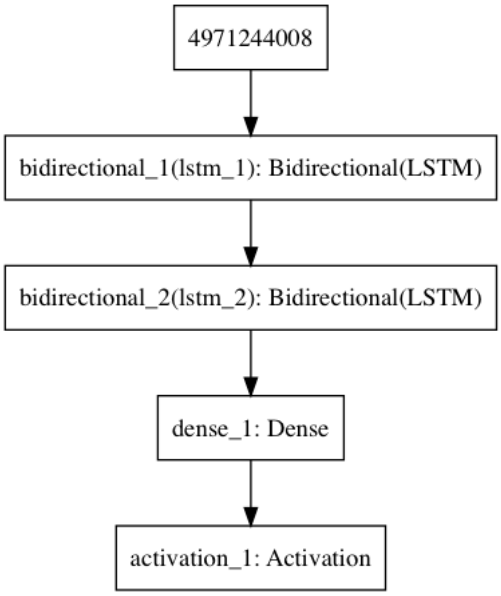

Figure 15: The LSTM model implemented in keras

The model is then compiled and trained against the training set. The training set is composed of prices sequences of a length of 7. After the model is trained, the model can now be used to predict prices given price sequences.

A trading strategy can now be created by using the model to predict prices. The model is fed the previous prices and if the model predicts a price higher than the current price, a buy order should be placed. Conversely, if the price is lower than the current price, a sell order should be placed.

```
for i in range(1, len(actual_prices)):
    if predicted_prices[i] > actual_prices[i - 1] and last_position ==
        Type.SELL:
        print('BUY @', actual_prices[i])
        last_position = Type.BUY
        stack /= actual_prices[i]

    if predicted_prices[i] < actual_prices[i - 1] and last_position ==
        Type.BUY:
        print('SELL @', actual_prices[i])
        last_position = Type.SELL
```

```
            stack *= actual_prices[i]
```

Code 10: Trading strategy using LSTM

4.6 Google trends

To find interest from search terms used by the Google search engine, the `pytrends` package is used. The package provides methods to find interest from various search terms. The `interest_over_time` method is used to return the interest in a normalized form where the values is between 0 and 100.

A trading strategy can be created by measuring the percent change between previous and current interest points. The direction of interest is used to indicate whether to place buy or sell signals.

```
        for i in range(1, n):

         curr = data['bitcoin'][i]
         prev = data['bitcoin'][i - 1]

         change = ((float(curr) - prev) / prev) * 100

         curr_date = datetimes[i].strftime("%d/%m/%Y")
         price = bitcoin_prices[(bitcoin_prices.Date ==
             curr_date)]['BTC_Close**'].values[0]

         if change > 10 and last_position == Type.SELL:
             print('BUY @', curr_date, price)
             last_position = Type.BUY
             stack /= price
             print(stack)

         if change < 10 and last_position == Type.BUY:
             print('SELL @', curr_date, price)
             last_position = Type.SELL
             stack *= price
             print(stack)
```

Code 11: Trading strategy using Google trends

4.7 New exchange additions

To scan for new cryptocurrencies to be added onto exchanges, a simple solution would be to make calls to exchange APIs and take notice of all the markets. When new cryptocurrencies are added to exchanges, new markets are created to allow traders to buy or sell the cryptocurrency that was added. The `ccxt` package in Python is a library for interacting with several cryptocurrency exchanges. It uses the same interface to interact with all exchanges which makes it easy to write code to interact with different exchanges.

The trading strategy can be defined by constantly scanning the markets provided by an exchange and finding the difference between the markets fetched previously and currently. This will find the new markets available if there is any. Then buy orders are executed on the new markets if found.

```
binance = ccxt.binance({
    'enableRateLimit': True,
})

binance.load_markets()

previous_tickers = binance.fetch_tickers()

while True:
    time.sleep(60)
    current_tickers = binance.fetch_tickers()
    diff = set(current_tickers.keys()) - set(previous_tickers.keys())

    if diff > 0:
        print(diff) #contains recent added markets
        #place_buy_order()
```

Code 12: Trading strategy using new exchange additions

4.8 Tether

To find Tether historical prices, the Coinmarket API is used. The trading strategy utilizing tether price can be defined by placing buy orders if the price is less than 1 and placing sell orders if the price is larger than 1.

```
    if tether_prices[i] < 1 and last_position == Type.SELL:
      print('BUY @', time_str)
      last_position = Type.BUY
      stack /= price

    if tether_prices[i] > 1 and last_position == Type.BUY:
      print('SELL @', time_str)
      last_position = Type.SELL
      stack *= price
```

Code 13: Trading strategy using Tether prices

4.9 Graphical User Interface

A graphical user interface is built to back-test the trading strategies mentioned above easily. The user interface should easily be able to interact with the Python scripts that provide the calculations in order to back-test the data. The user interface is therefore built as a web application to provide a front-end for the trading strategies.

The back-end of the web application is built on `Flask` which is a python library for creating simple REST API. The REST API will provide endpoints of running the trading strategies on the data and returning the data given to plot the results. The trading strategies are imported to be used in the server.

```
@app.route("/ma")
@cross_origin()
def ma():
    collection_name = request.args.get('collection_name')
    ma_type = request.args.get('ma_type')
    data = {}
    if ma_type == 'Simple':
        candlestick, ma = sma.get_sma(collection_name)
    if ma_type == 'Weighted':
        candlestick, ma = wma.get_wma(collection_name)
    if ma_type == 'Exponential':
        candlestick, ma = ema.get_ema(collection_name)

    ma = replace_nan_with_none(ma)
```

```
    data['candlestick'] = candlestick
    data['ma'] = ma
    return jsonify(data)
```

Code 14: Flask endpoint for getting moving averages

The web application uses HTML, CSS, and Javascript to power the frontend of the web page. Bootstrap is used as a frontend library to quickly create the user interface as it provides many pre-built user interface components. The user has options to select a trading strategy. Once a trading strategy is selected, an AJAX request is sent to the server returning with data which is shown to the user. The data that is shown to the user is a visualized chart of the strategy.

Figure 16: The graphical user interface

5 Experiments and Evaluation

This section describes the experiments that were undertaken. The experiments will start with a stack of bitcoin. The trading strategy will then make buy or sell trades. In the end, profit is made if the bitcoin stack is larger than it was started with. The profit is shown as percent change. A positive value would result in a profit and a negative value would result in a loss.

5.1 Arbitrage

From the exchange arbitrage script, a total of 536 arbitrage opportunities was created from 26 different markets.

Cryptocurrency Used For Arbitrage	**Arbitrage Opportunities**
ICX	21
VET	32
TRX	57
NEO	2
BNB	15

Table 1: Tri-arbitrage opportunities within 24 hours of running Python script

The results have shown that there are indeed numerous arbitrage opportunities that still exist in the market. The profits may be small when making a successful trade, however, it is a risk-free opportunity if correct calculations are done in finding the ask and buy prices.

Exchange arbitrage works in theory but it is very unlikely it works in practice as the experiment does not take into account the transferring time and fees of cryptocurrencies between exchanges. Tri-arbitrage is, therefore, a much better option when using arbitrage to make a profit.

5.2 Technical Analysis

The data that was used in these experiments contains candlestick data that was recorded on the BTCUSDT market from the Binance exchange. The candlestick data was collected from the dates of 19th June 2018 to 15th July 2018.

Testing Strategy	BTCUSDT_1M	BTCUSDT_5M	BTCUSDT_30M	BTCUSDT_1H	BTCUSDT_4H	BTCUSDT_1D
SMA	-1.76	-0.66	2.97	17.30	9.01	32.69
WMA	-0.38	0.62	6.19	7.28	13.1	24.9
EMA	-1.87	3.05	8.27	8.14	11.78	20.83
BBands	-3.94	3.85	16.89	9.19	9.84	8.42
MACD	-1.96	3.71	14.77	16.81	0.33	N/A (Not Enough Data)
RSI	-1.72	2.98	12.64	10.40	0	22.39

Table 2: Results of backtesting the technical analysis trading strategies on different time frames of the BTCUSDT market

From the results recorded in table 3, it is clear that larger profit is made by using larger timeframes and using short-term timeframes resulted in some losses. All trading indicators seemed to have performed similarly.

5.3 Sentiment Analysis

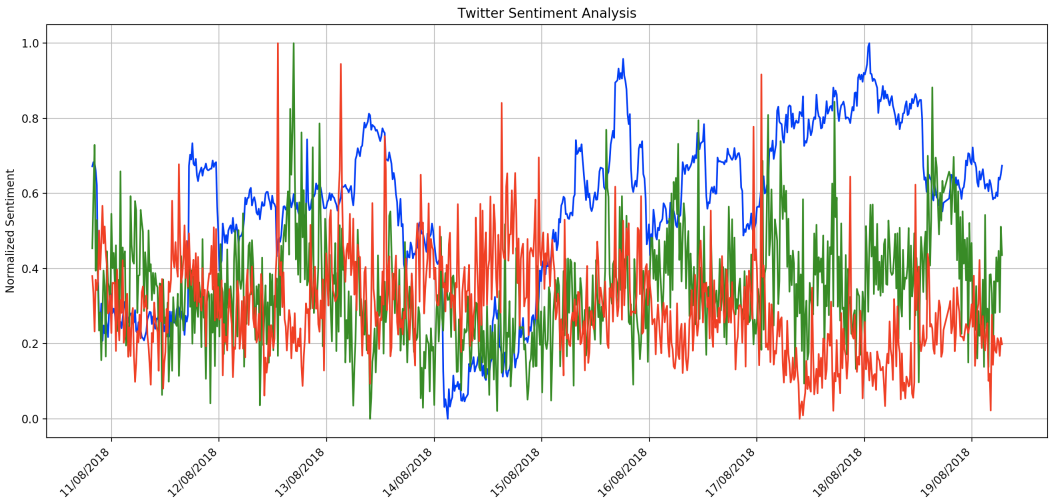

Figure 17: Sentiment analysis of twitter tweets. Blue: Bitcoin price, Red: Negative sentiment, Green: Positive sentiment

The data collected from the twitter tweets were at intervals of 15 minutes. The data was collected from 11 August 2018 to 19 August 2018. The sentiment and bitcoin price was recorded. The data proved to be very noisy and difficult to interpret. Looking closely, when Bitcoin's price seems to increase, the negative sentiment seems to go lower than the positive sentiment and vice versa.

Sentiment	**Profit**
Positive	-2.38
Negative	-2.02

Table 3: Profit shown using sentiment analysis

Using the sentiment to trigger buy and sell orders of bitcoin, the strategy failed to make a profit as using both positive and negative made losses. Both sentimental values seemed to give a similar loss as shown in the table above.

5.4 Long short-term memory

The data used in this experiment was Bitcoin's historical price. The data range from 2015 to the current price. The long short-term memory was given the following parameters:

Property	**Value**
Neurons	512
Epochs	5
Batch size	12
Activation Function	tanh
Loss	mse (mean squared error)
Optimizer	adam
Metric	mae (mean absolute error)
Price Sequence Length	7

Table 4: Settings used to train the model

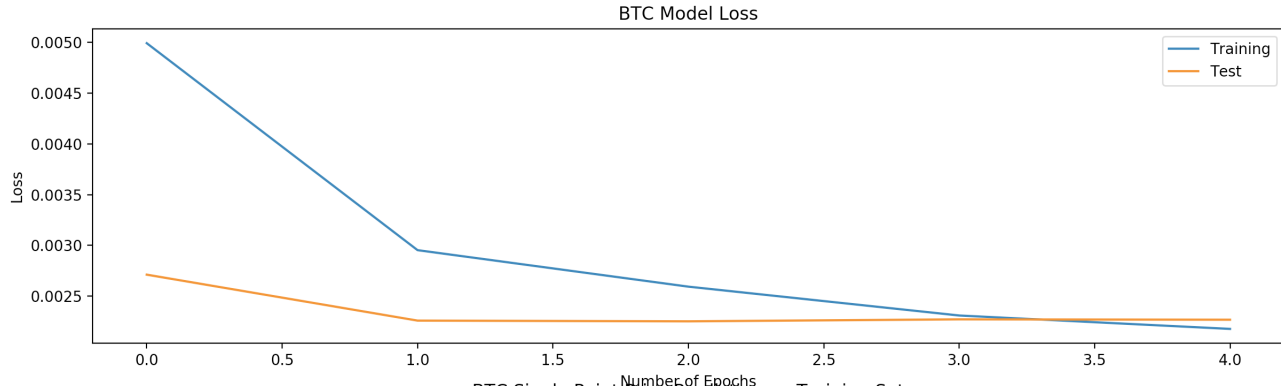

Figure 18: Model loss as the number of epochs increase

The training phase had 5 iterations which are shown by the number of epochs. The loss started off very high in both the training and testing set and decreased as the number of epochs went on.

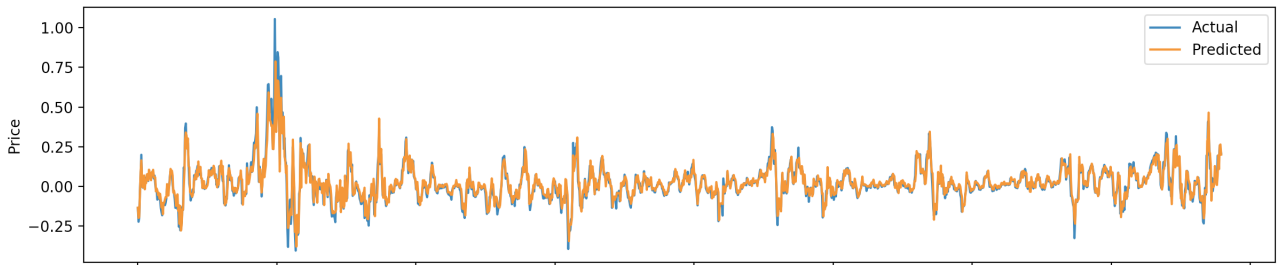

Figure 19: Actual and predicted price on the training set

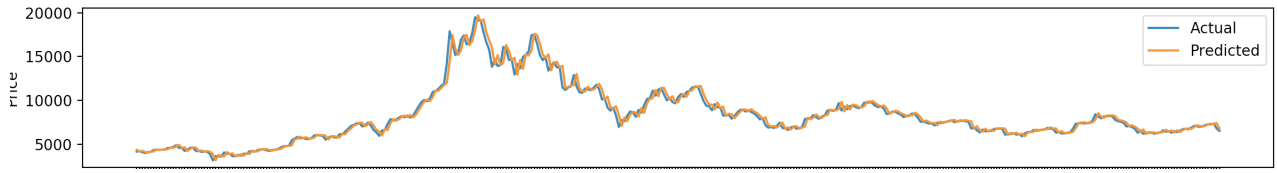

Figure 20: Actual and predicted price on the testing set

The model seemed to have predicted extremely well as the values of the actual and predicted prices are very similar. The back-test of the trading strategy on the test set resulted in a profit of 13.54%.

5.5 Google trends

The data used in this experiment was provided by Google trends. The data used is from 2014 to 2018.

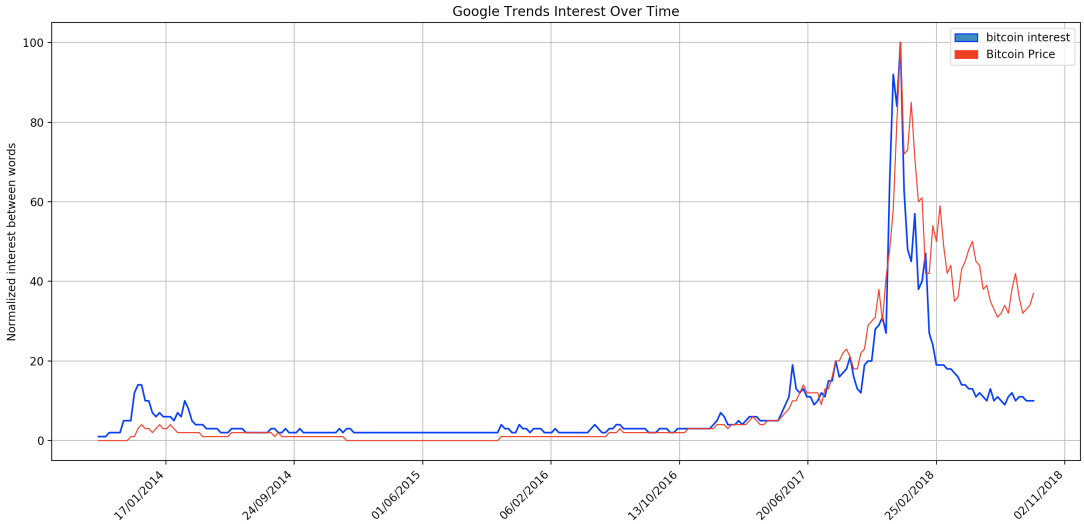

Figure 21: Google interest of the search term "bitcoin" vs the price of Bitcoin

Search Term	**Profit by backtest**
bitcoin	1099%
cryptocurrency	574%
blockchain	653%

Table 5: Results from back-testing google trend interest on various search terms

The data shown in figure 21 shows that the price and interest of bitcoin are very much correlated. Using the interest to influence buying and selling, the trading strategy did extremely well as large profits were made on the three search terms mentioned in the table above. However, this was due to the data ranging from early Bitcoin years as large profit could only be made if Bitcoin was brought early.

5.6 New exchange additions

The addition of cryptocurrency in exchanges could not be tested as, during the running of the script, no new cryptocurrency additions happened in the Binance exchange which was being tracked.

5.7 Tether

The data used in tether is the price data which ranges from May 2017 to September 2018.

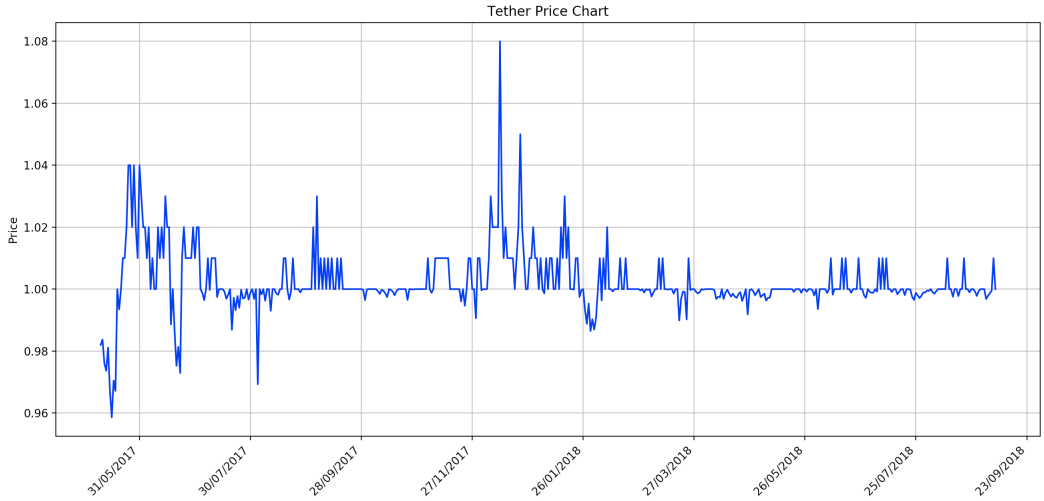

Figure 22: Tether price chart

From the price chart, it is evident that Tether is not pegged at one dollar at all times. The trading strategy took advantage of this data to trade bitcoin and the result performed by the back-test resulted in a profit of 4.19%.

5.8 Evaluation

The experiments performed analyzed several trading strategies which utilize several different variables to make a profit by making buy or sell trades. Whether the trading strategy did or did not make a profit, the performance of the trading strategy is explored on the cryptocurrency market.

The trading strategies in the paper are back-tested using historical data. No actual trading has been done by using these strategies so therefore in a live setting, these strategies may perform differently as there are many unaccounted scenarios that could take place.

The results from back-testing the trading strategies may change from supplying different data to back-test. This is one important aspect to consider as different markets will contain different trends and patterns.

6 Conclusion

6.1 Summary

The cryptocurrency marking is proven to be a wild market with very rapid price movements. In the market, a large amount of profit can be made within the market as the high volatility attracts many traders.

With the use of computer programs, automated trading strategies can be created and from the back-test results conducted in the experiments in this paper, it is shown that profit can be made. Trading strategies in the cryptocurrency market still work even with the constant price manipulation happening.

Automated trading provides many advantages over human trading and creates opportunities for profit that are not seen when trading manually. The use of automated trading is becoming much more popular and therefore introduces much more competition between trading strategies. The state of the art trading strategies in the future will be the ones that will be able to make the most profit and beat the competition.

6.2 Future work

There many other trading indicators and variables that can be used to create trading strategies with. In the paper, a small number of trading strategies and therefore in future work, more trading strategies can be explored.

The data in the cryptocurrency market is huge. There exist thousands of cryptocurrency markets with data on these markets being available. The trading strategies can be back-tested with more data to find out performance from using data in several markets. This would give a better measurement of performance.

The trading strategies were not executed on live data, therefore future work could include running the trading strategy by live market data by continuously capturing and calculating the required data to be used by the trading strategies.

References

[1] Igor Makarov and Antoinette Schoar. *Trading and Arbitrage in Cryptocurrency Markets* - London School of Economics. 2018.

[2] Kai Sedgwick. *Do Crypto Trading Bots Really Work?*. 2018. https://news.bitcoin.com/do-crypto-trading-bots-really-work/

[3] *Market Cycles in Cryptocurrency*. 2018. https://cryptocurrencyfacts.com/market-cycles-in-cryptocurrency/

[4] Arthur Iinuma. *Why Is the Cryptocurrency Market So Volatile: Expert Take*. 2018. https://cointelegraph.com/news/why-is-the-cryptocurrency-market-so-volatile-expert-take

[5] Anton Badev and Matthew Chen. *Bitcoin: Technical Background and Data Analysis* - Finance and Economics Discussion Series Divisions of Research & Statistics and Monetary Affairs Federal Reserve Board, Washington, D.C. 2014.

[6] *How to Backtest a Trading Strategy*. 2018. https://tradingstrategyguides.com/how-to-backtest-a-trading-strategy/

[7] *Maker vs. Taker in Cryptocurrency*. 2018. https://cryptocurrencyfacts.com/maker-vs-taker-cryptocurrency/

[8] Robert L Kissell. *Algorithmic trading strategies* - Fordham University. 2006.

[9] Darryn Pollock. *The Mess That Was Mt. Gox: Four Years On*. 2018. https://cointelegraph.com/news/the-mess-that-was-mt-gox-four-years-on

[10] Alex Heid. *Analysis of the Cryptocurrency Marketplace*. 2013. http://www.hackmiami.org/whitepapers/HackMiami-Analysis_of_the_Cryptocurrency_Marketplace.pdf

[11] TeamToken24. *Common Order Types for Cryptocurrency Trading*. 2018. https://www.tokens24.com/cryptopedia/trading/common-order-types-cryptocurrency-trading

[12] askmike. *[ARBITRAGE] Agent Smith: Arbing the great bull run of 2017*. 2018. https://forum.gekko.wizb.it/thread-56746.html

[13] Daniel Faggella. *Machine Learning in Finance Present and Future Applications.* 2018. https://www.techemergence.com/machine-learning-in-finance/

[14] Apoorv Agarwal, Boyi Xie, Ilia Vovsha, Owen Rambow, Rebecca Passonneau. *Sentiment Analysis of Twitter Data* - Columbia University. 2018. http://delivery.acm.org/10.1145/2030000/2021114/p30-agarwal.pdf?ip=86.23.19.162&id=2021114&acc=OPEN&key=4D4702B0C3E38B35%2E4D4702B0C3E38B35%2E4D4702B0C3E38B35%2E6D218144511F3437&__acm__=1536321012_e257f55bc19eab8470aa206c13cb9194

[15] Sepp Hochreiter and Jrgen Schmidhuber. *Long Short-Term Memory.* 1997.

[16] Jiarui Ni, Chengqi Zhang. *An Efficient Implementation of the Backtesting of Trading Strategies.* 2005.

[17] Neil Gandal, Hanna Halaburda. *Competition in the Cryptocurrency Market.* 2013.

[18] Vadim Avdeychik, Justin Capozzi. *SECs Division of Investment Management voices concerns over registered funds investing in cryptocurrencies and cryptocurrency-related products.* 2018.

[19] David Garcia, Frank Schweitzer. *Open Access Social signals and algorithmic trading of Bitcoin.* 2015. http://rsos.royalsocietypublishing.org/content/2/9/150288

[20] Janick Rohrbach, Silvan Suremann, Joerg Osterrieder. *Momentum and trend following trading strategies for currencies and bitcoin* - Zurich University of Applied Sciences, School of Engineering. 2017.

[21] Peter M. Krafft, Nicolas Della Penna, Alex Sandy Pentland. *An Experimental Study of Cryptocurrency Market Dynamics.* 2018.

[22] Irene Aldridge. *A Practical Guide to Algorithmic Strategies and Trading Systems.* 2009.

[23] Sanford J. Grossman. *An Analysis of the Implications for Stock and Futures Price Volatility of Program Trading and Dynamic Hedging Strategies.* 1987.

[24] Laura Alessandretti, Abeer ElBahrawy, Luca Maria Aiello, Andrea Baronchelli. *Machine Learning the Cryptocurrency Market* - 'City, University of London. 2018. https://arxiv.org/pdf/1805.08550.pdf